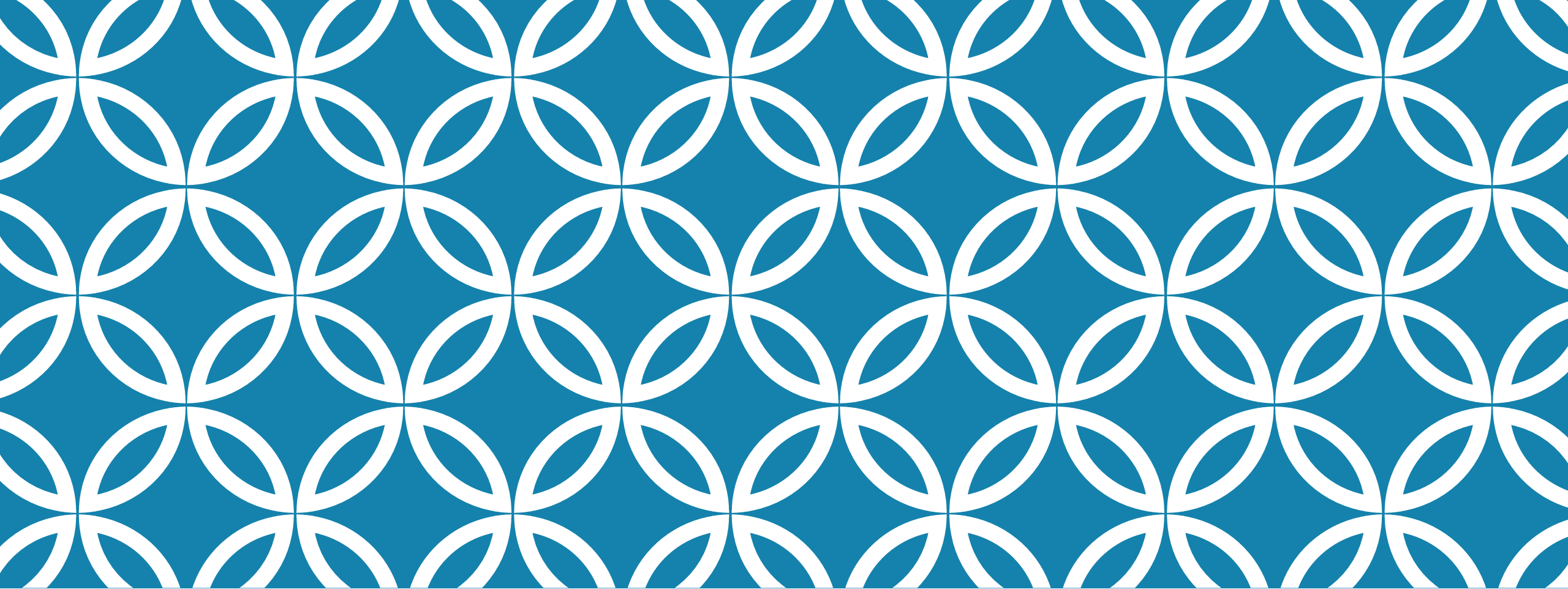




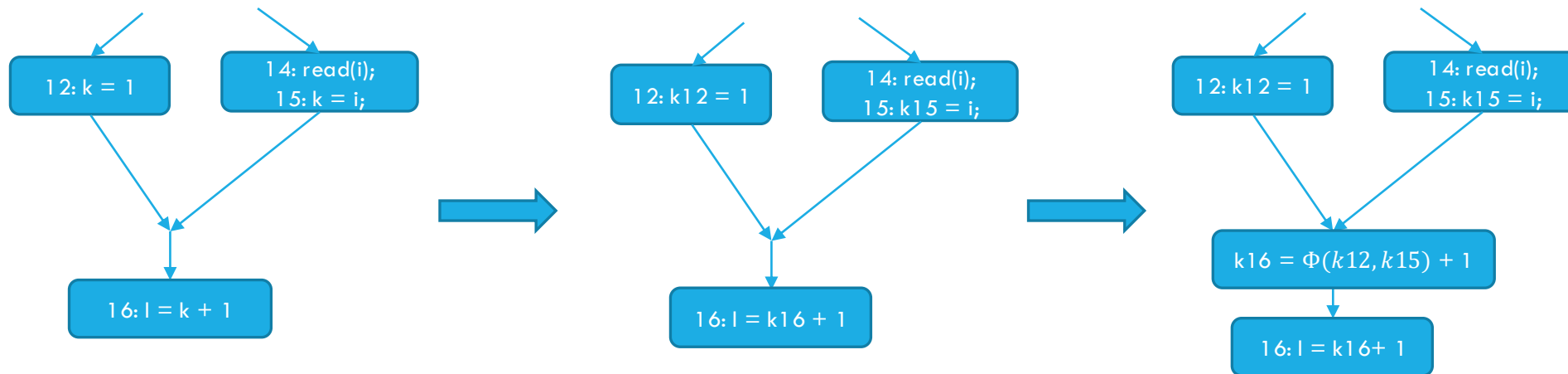
STATIC PROGRAM ANALYSIS

Lecture 13 – SSA vs Original
representations

SINGLE STATIC ASSIGNMENT

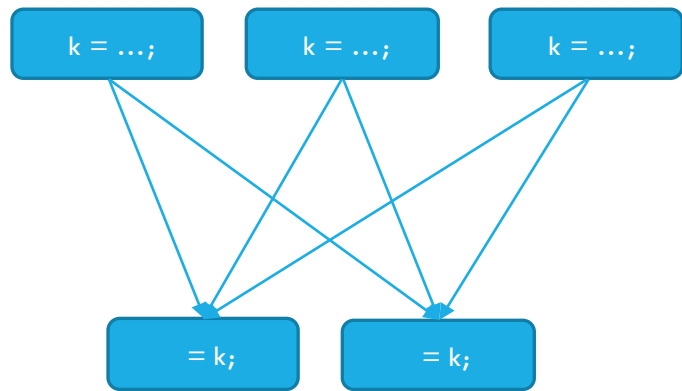
Mainly used in compiler optimization analysis

Encodes Use-Def information explicitly through Φ nodes

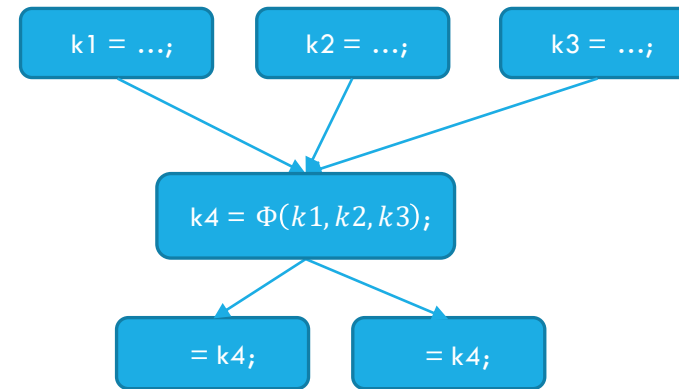


ADVANTAGE OF SSA IR

SSA reduces the representation complexity of use-defs chains (but requires more computation for it)

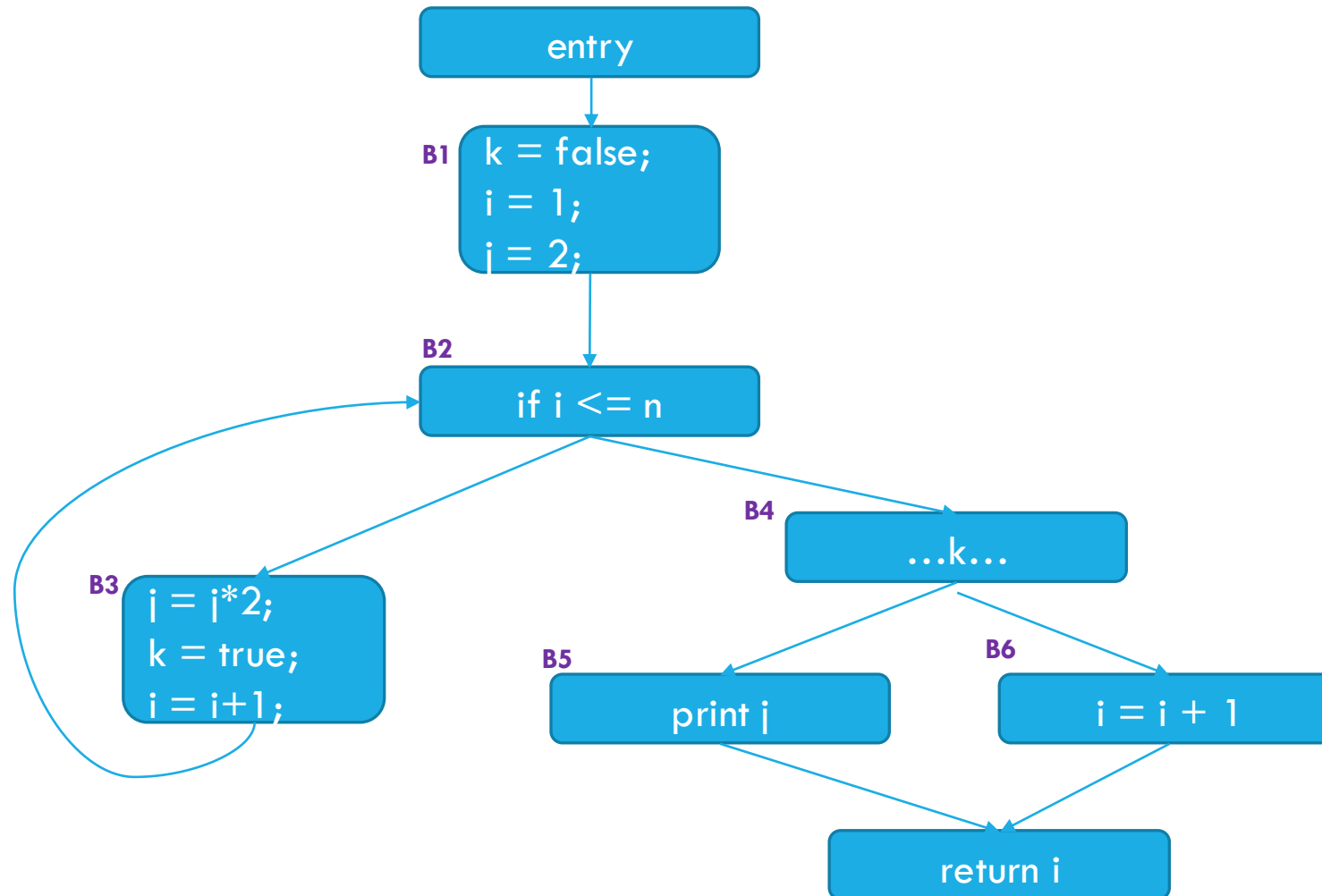


$$|Defs| * |Uses|$$



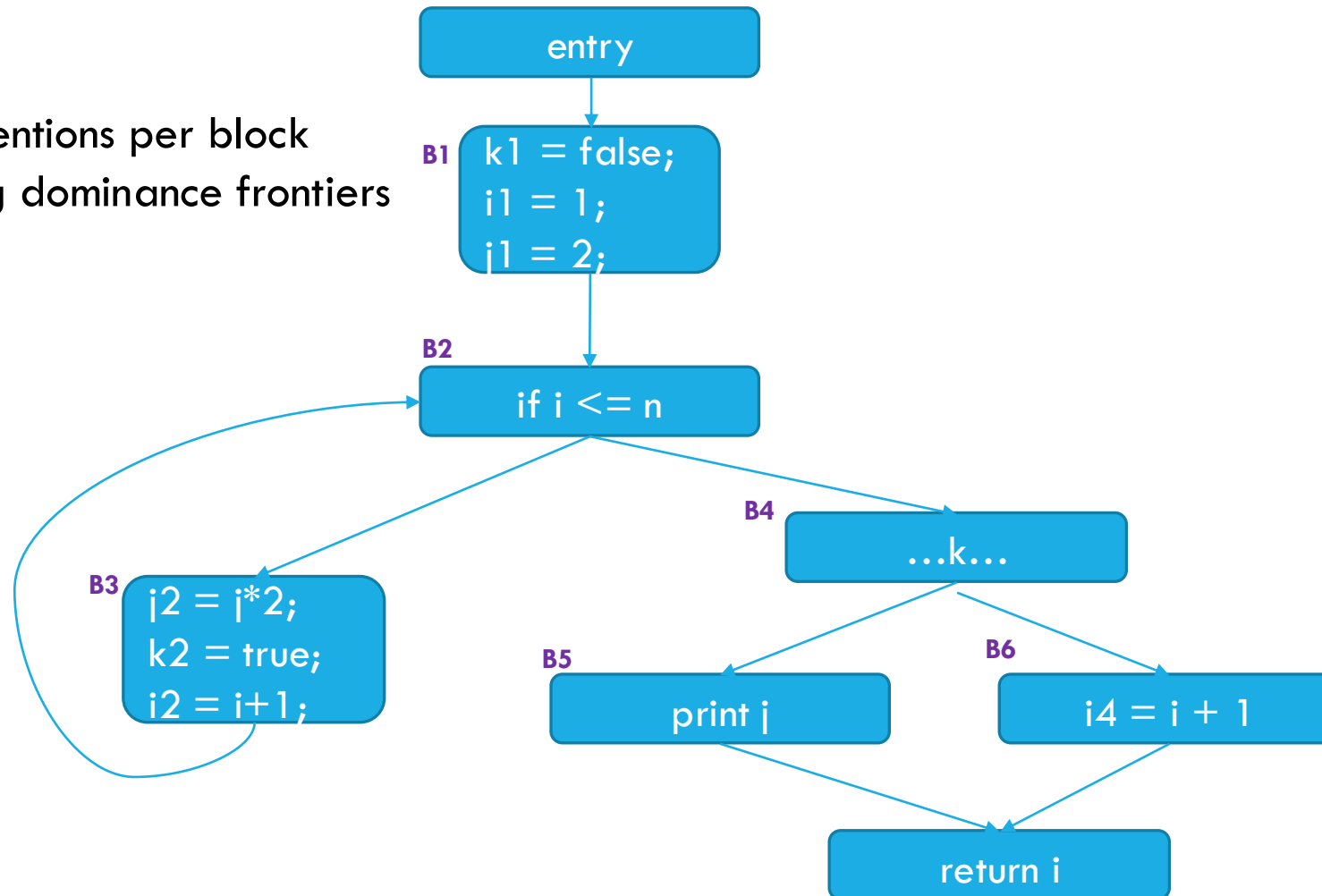
$$|Defs| + |Uses|$$

SSA EXAMPLE WITH A LOOP

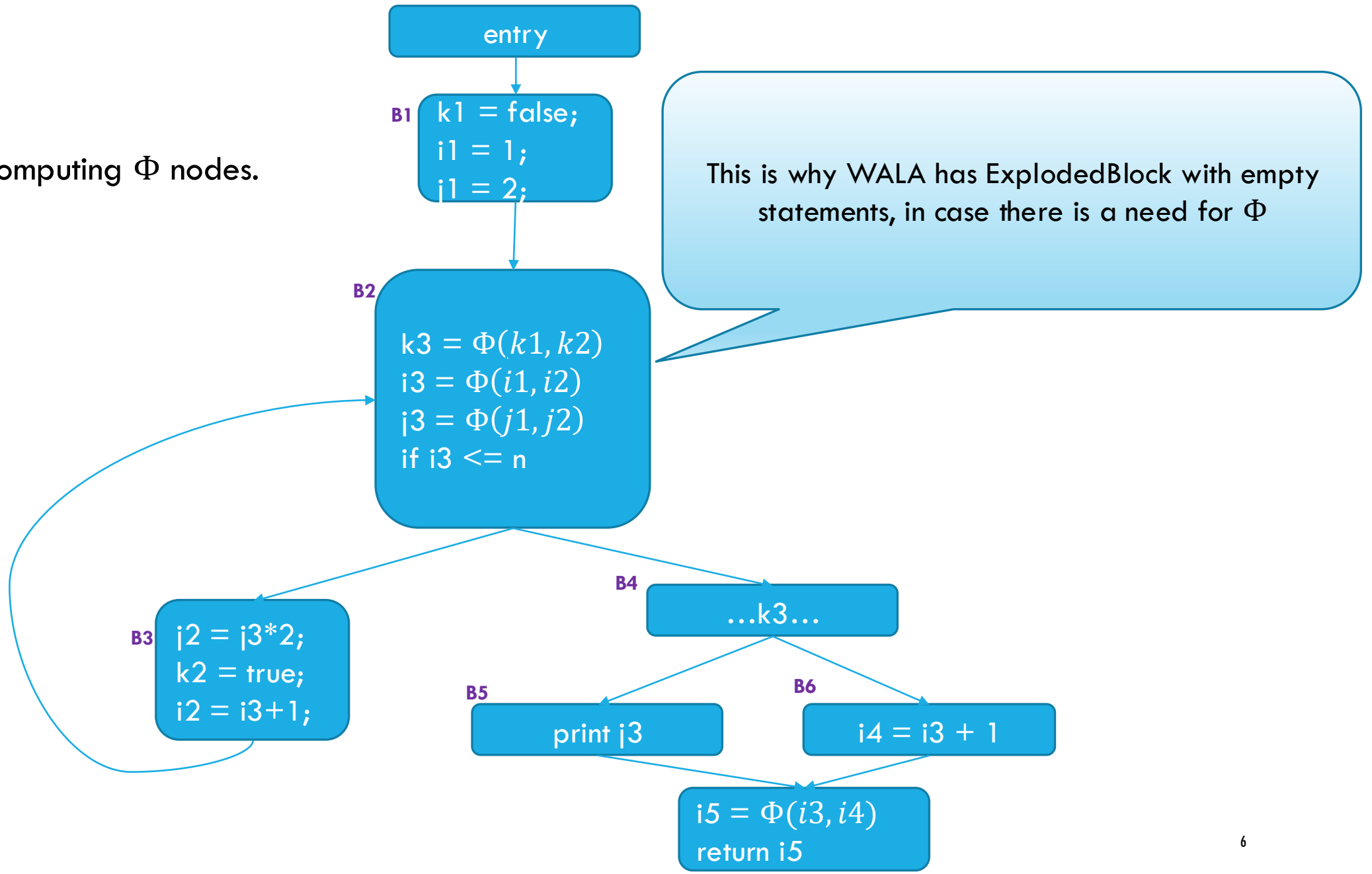


SSA EXAMPLE WITH A LOOP

Renaming detentions per block
and computing dominance frontiers



Computing Φ nodes.



COMPUTING CLASSIC ANALYSIS

Regular representation

- Iterate $\{i@1, i@2\}$ values until a fixed point is reached
- At each statement, compare the RD set and variables in rhs

SSA representation

Reaching definitions:

- Each definition has unique value: $i1, i3, i4$
- Iterate over code and collect all rhs
 - $\{i3, i5\}$
 - Using Phi nodes Φ augment the set $\{i1, i2, i3, i4, i5\}$
 - If a definition is not present in the set then unused definition



PHI NODES IN WALA

Let's revisit our RD analysis